

Introduction to Electronic Design Automation

Homework 2

(Problems 1-5)

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1 Cofactor

1. (a)

The equalities does not hold for u and v is a literal for the same variable. For example, let $f(x) = x$, $u = x$, and $v = \neg x$. Then, $f_u = 1$ and $(f_u)_v = 1$, but $f_v = 0$ and $(f_v)_u = 0$.

1. (b)

Let v be a literal of variable x . If $v = x$, then

$$(\neg f)_v = (\neg f)|_{x=1} = \neg(f|_{x=1}) = \neg(f_v)$$

If $v = \neg x$, then

$$(\neg f)_v = (\neg f)|_{x=0} = \neg(f|_{x=0}) = \neg(f_v)$$

Hence, $(\neg f)_v = \neg(f_v)$ for any literal v .

1. (c)

Let v be a literal of variable x . By Shannon expansion,

$$f = xf_x + \neg x f_{\neg x}$$

and

$$g = xg_x + \neg x g_{\neg x}.$$

We first prove the case $v = x$. Using Shannon expansion on the function $f \oplus g$, we have

$$f \oplus g = x(f \oplus g)_x + \neg x(f \oplus g)_{\neg x}.$$

On the other hand, substituting the Shannon expansions of f and g into $f \oplus g$, we get

$$f \oplus g = (xf_x + \neg x f_{\neg x}) \oplus (xg_x + \neg x g_{\neg x}).$$

When $x = 1$, this becomes

$$(f \oplus g)_v = (f \oplus g)|_{x=1} = f_x \oplus g_x$$

Next, when $v = \neg x$, setting $x = 0$ in the same expansion gives

$$(f \oplus g)_v = (f \oplus g)|_{x=0} = f_{\neg x} \oplus g_{\neg x} = f_v \oplus g_v.$$

Therefore, in both cases,

$$(f \oplus g)_v = f_v \oplus g_v.$$

Hence $(f \oplus g)_v = (f_v) \oplus (g_v)$ is true.

2 Quantified Boolean Formula

By De Morgan's law, we have

$$\neg(\exists x f(x)) = \forall x(\neg f(x))$$

$$\neg(\forall x f(x)) = \exists x(\neg f(x))$$

2. (a)

For $\Rightarrow$, we have

$$\neg(\forall x, \exists y. f(x, y, z)) \equiv \exists x. \neg(\exists y. f(x, y, z)) \equiv \exists x. \forall y. \neg f(x, y, z).$$

For $\Leftarrow$, we have

$$\exists x. \forall y. \neg f(x, y, z) \equiv \neg(\forall x. \neg(\forall y. \neg f(x, y, z))) \equiv \neg(\forall x. \exists y. f(x, y, z)).$$

Therefore, both directions hold, and hence the equivalence is true.

2. (b)

For $(\Rightarrow)$, consider the following counterexample:

$$f(x, y, z) = (x \wedge y) \vee (x \wedge \neg y).$$

Then $\exists x. \forall y. f(x, y, z)$ is true, since by choosing $x = 1$, we have

$$f(1, y, z) = (1 \wedge y) \vee (1 \wedge \neg y) = y \vee \neg y = 1$$

for all y .

However, $\forall x. \exists y. f(x, y, z)$ is false, since when $x = 0$, we have

$$f(0, y, z) = (0 \wedge y) \vee (0 \wedge \neg y) = 0$$

for every y . Hence, there is no value of y that makes f true.

For $(\Leftarrow)$, consider the following counterexample:

$$f(x, y, z) = (x \wedge y) \vee (\neg x \wedge \neg y).$$

Then $\forall x. \exists y. f(x, y, z)$ is true, since:

- when $x = 1$, choosing $y = 1$ makes f true;
- when $x = 0$, choosing $y = 0$ makes f true.

However, $\exists x. \forall y. f(x, y, z)$ is false, since:

- if $x = 1$, then $f(x, y, z) = y$, which is not true for all y ;
- if $x = 0$, then $f(x, y, z) = \neg y$, which is not true for all y .

Hence, neither direction holds in general.

2. (c)

For $(\Rightarrow)$, assume $\exists x. \forall y. f(x, y, z)$ is true. Then there exists some fixed x such that $f(x, y, z)$ is true for all y . Let this x be s . Then, for every y , we have $f(s, y, z)$ is true. Hence, for every y , we can choose that same $x = s$, so $\forall y. \exists x. f(x, y, z)$ is true.

For $(\Leftarrow)$, consider the following counterexample:

$$f(x, y, z) = (x \wedge y) \vee (\neg x \wedge \neg y).$$

Then $\forall y. \exists x. f(x, y, z)$ is true, since:

- when $y = 1$, choose $x = 1$;

- when $y = 0$, choose $x = 0$.

However, $\exists x. \forall y. f(x, y, z)$ is false, since:

- if $x = 1$, then $f(x, y, z) = y$, which is not true for all y ;
- if $x = 0$, then $f(x, y, z) = \neg y$, which is not true for all y .

Hence, only $(\Rightarrow)$ holds in general.

2. (d)

For $(\Rightarrow)$, we use the following implication steps:

1. $\forall x. (f(x, y) \wedge g(x, z))$ (premise)
2. $f(u, y) \wedge g(u, z)$ (UI, 1, with $x = u$ is arbitrary)
3. $f(u, y)$ (simplification, 2)
4. $\forall x. f(x, y)$ (UG, 3)
5. $g(u, z)$ (simplification, 2)
6. $\forall x. g(x, z)$ (UG, 5)
7. $\forall x. f(x, y) \wedge \forall x. g(x, z)$ (conjunction, 4 and 6)

Hence, $\forall x. (f(x, y) \wedge g(x, z))$ implies $\forall x. f(x, y) \wedge \forall x. g(x, z)$.

For $(\Leftarrow)$, we use the following implication steps:

1. $\forall x. f(x, y) \wedge \forall x. g(x, z)$ (premise)
2. $\forall x. f(x, y)$ (simplification, 1)
3. $f(u, y)$ (UI, 2, with $x = u$ is arbitrary)
4. $\forall x. g(x, z)$ (simplification, 1)
5. $g(u, z)$ (UI, 4, with $x = u$ is arbitrary)
6. $f(u, y) \wedge g(u, z)$ (conjunction, 3 and 5)
7. $\forall x. (f(x, y) \wedge g(x, z))$ (UG, 6)

Hence, $\forall x. f(x, y) \wedge \forall x. g(x, z)$ implies $\forall x. (f(x, y) \wedge g(x, z))$. Thus, the equivalence is true.

2. (e)

For $(\Rightarrow)$, consider the following counterexample:

$$f(x, y) = x \vee y \quad \text{and} \quad g(x, z) = \neg x \vee z$$

Then $\forall x. (f(x, y) \vee g(x, z))$ is true, since

$$f(x, y) \vee g(x, z) = (x \vee y) \vee (\neg x \vee z) = x \vee \neg x \vee y \vee z = 1 \vee y \vee z = 1$$

for every x . However, $\forall x. f(x, y) \vee \forall x. g(x, z)$ is false, since consider $x = 0$, we have $f(x, y) = y$ and $g(x, z) = z$, which are not true for all y and z .

For $(\Leftarrow)$:

Assume

$$\forall x. f(x, y) \vee \forall x. g(x, z).$$

Case 1:

$$\forall x. f(x, y)$$

Then for arbitrary u :

$$f(u, y)$$

Hence

$$f(u, y) \vee g(u, z)$$

Thus

$$\forall x.(f(x, y) \vee g(x, z)).$$

Case 2:

$$\forall x.g(x, z)$$

Similarly:

$$\forall x.(f(x, y) \vee g(x, z)).$$

Therefore,

$$\forall x.f(x, y) \vee \forall x.g(x, z) \rightarrow \forall x.(f(x, y) \vee g(x, z)).$$

Hence, only $(\Leftarrow)$ holds in general.

3 Boolean Function Representation

Consider the majority function $f(x_1, x_2, x_3, x_4, x_5) = (x_1 + x_2 + x_3 + x_4 + x_5 \geq 3)$.

3. (a)

The K-map of f :

• **Case 1:** $x_5 = 1$.

		x_3x_4			
		00	01	11	10
x_1x_2	00	0	0	1	0
	01	0	1	1	1
	11	1	1	1	1
	10	0	1	1	1

• **Case 2:** $x_5 = 0$.

		x_3x_4			
		00	01	11	10
x_1x_2	00	0	0	0	0
	01	0	0	1	0
	11	0	1	1	1
	10	0	0	1	0

Hence, the minimal sum-of-products form of f is

$$\begin{aligned} f = & x_1x_2x_3 + x_1x_2x_4 + x_1x_2x_5 \\ & + x_1x_3x_4 + x_1x_3x_5 + x_1x_4x_5 \\ & + x_2x_3x_4 + x_2x_3x_5 + x_2x_4x_5 \\ & + x_3x_4x_5. \end{aligned}$$

3. (b)

By turning the minimal sum-of-products form into a product-of-sums form, we have

$$\begin{aligned} f = & (x_1 + x_2 + x_3)(x_1 + x_2 + x_4)(x_1 + x_2 + x_5) \\ & (x_1 + x_3 + x_4)(x_1 + x_3 + x_5)(x_1 + x_4 + x_5) \\ & (x_2 + x_3 + x_4)(x_2 + x_3 + x_5)(x_2 + x_4 + x_5) \\ & (x_3 + x_4 + x_5). \end{aligned}$$

which is the minimal product-of-sums form of f .

3. (c)

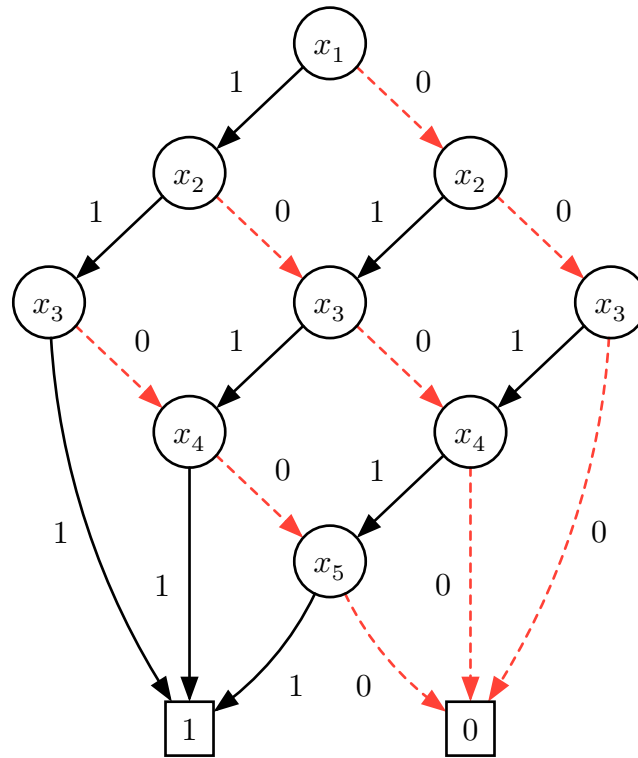


Figure 1: The ROBDD of f .

3. (d)

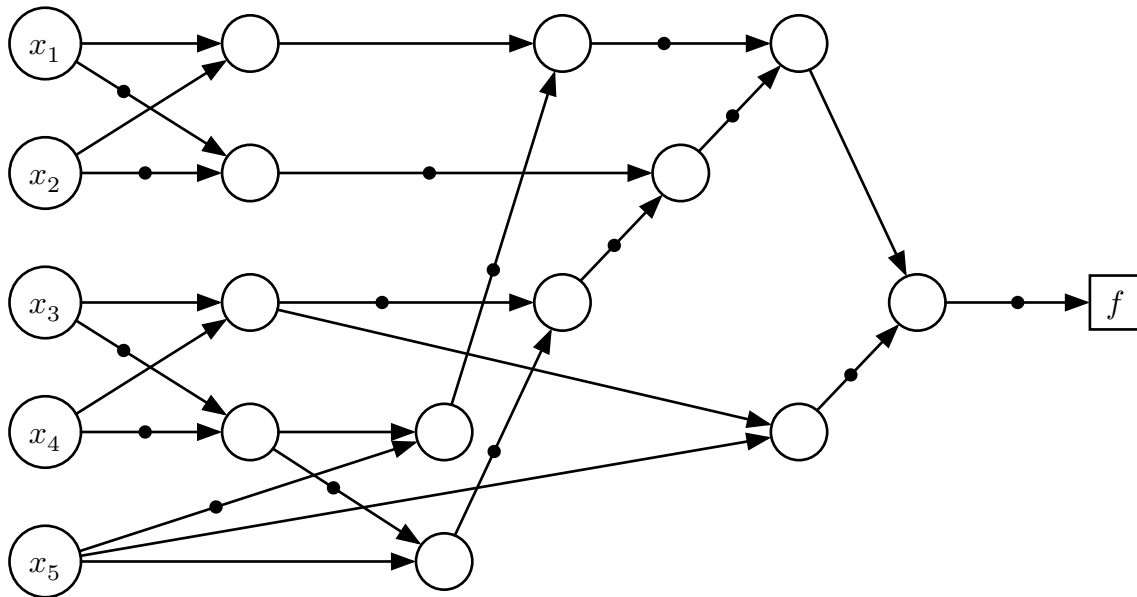


Figure 2: The AIG of f . Use total 12 nodes.

3. (e)

We can express f as:

$$f = M(x_1, M(x_2, x_3, x_5), M(x_4, x_5, M(x_1, x_2, x_3)))$$

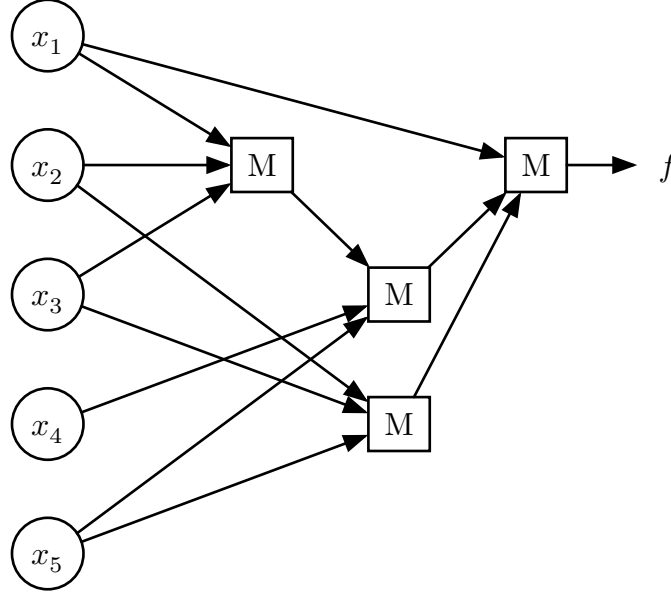


Figure 3: The circuit of f using 3-input majority gates. Use total 4 gates.

4 Application of Quantified Boolean Formula

4. (a)

First of all, this game only has five stones, so it must terminate within at most five rounds. Hence, we can encode the winning condition of player 1 as the following QBF:

$$\exists M_1. \forall M_2. \exists M_3. \forall M_4. \exists M_5. \quad \Phi$$

where M_t denotes the Boolean variables describing the move at round t , and Φ is a Boolean formula encoding legality, state transition, and winning condition.

We now define the move variables. Let $m_{t,i,a}$ denote “at round t , the player removes a stones from pile i ”. The possible moves are:

$$m_{t,1,1}, m_{t,1,2}, m_{t,2,1}, m_{t,2,2}, m_{t,3,1}$$

Thus, M_t is simply a shorthand for the set of move variables

$$\{m_{t,1,1}, m_{t,1,2}, m_{t,2,1}, m_{t,2,2}, m_{t,3,1}\}.$$

Since each round consists of exactly one move, we impose a one-hot constraint:

$$\text{ONE}_t = (m_{t,1,1} \vee m_{t,1,2} \vee m_{t,2,1} \vee m_{t,2,2} \vee m_{t,3,1}) \\ \bigwedge_{x \neq y \in M_t} \neg(x \wedge y)$$

which enforces that exactly one move is chosen at round t .

Next, we encode the state. Instead of using integers, we use Boolean variables to represent whether each stone is still present. Let

$$a_1^t, a_2^t, b_1^t, b_2^t, c^t$$

denote the existence of each stone after round t . Here, a_1, a_2 are the two stones in pile 1, b_1, b_2 are the two stones in pile 2, and c is the unique stone in pile 3.

Initially, all stones are present:

$$I = a_1^0 \wedge a_2^0 \wedge b_1^0 \wedge b_2^0 \wedge c^0$$

To avoid invalid encodings, we enforce monotonicity within each pile:

$$O = \bigwedge_{t=0}^5 (a_2^t \rightarrow a_1^t) \\ \bigwedge_{t=0}^5 (b_2^t \rightarrow b_1^t)$$

This ensures that the second stone in a pile cannot exist if the first stone has already been removed. Hence the only valid states are those where the stones in each pile are removed from the second stone to the first stone.

Next, we ensure move legality. A move can only be applied if enough stones remain:

$$L_t = (m_{t,1,1} \rightarrow a_1^{t-1}) \\ (m_{t,1,2} \rightarrow a_2^{t-1}) \\ (m_{t,2,1} \rightarrow b_1^{t-1}) \\ (m_{t,2,2} \rightarrow b_2^{t-1}) \\ (m_{t,3,1} \rightarrow c^{t-1})$$

Now we define the state transition. Let T_t encode how the state changes after round t . The chosen pile loses stones, while the others remain unchanged.

$$T_t = (m_{t,1,1} \rightarrow ((a_2^{t-1} \rightarrow (a_1^t \wedge \neg a_2^t)) \wedge (\neg a_2^{t-1} \rightarrow (\neg a_1^t \wedge \neg a_2^t)) \wedge (b_1^t \leftrightarrow b_1^{t-1}) \wedge (b_2^t \leftrightarrow b_2^{t-1}) \wedge (c^t \leftrightarrow c^{t-1}))) \\ \wedge (m_{t,1,2} \rightarrow ((\neg a_1^t \wedge \neg a_2^t) \wedge (b_1^t \leftrightarrow b_1^{t-1}) \wedge (b_2^t \leftrightarrow b_2^{t-1}) \wedge (c^t \leftrightarrow c^{t-1}))) \\ \wedge (m_{t,2,1} \rightarrow ((b_2^{t-1} \rightarrow (b_1^t \wedge \neg b_2^t)) \wedge (\neg b_2^{t-1} \rightarrow (\neg b_1^t \wedge \neg b_2^t)) \wedge (a_1^t \leftrightarrow a_1^{t-1}) \wedge (a_2^t \leftrightarrow a_2^{t-1}) \wedge (c^t \leftrightarrow c^{t-1}))) \\ \wedge (m_{t,2,2} \rightarrow ((\neg b_1^t \wedge \neg b_2^t) \wedge (a_1^t \leftrightarrow a_1^{t-1}) \wedge (a_2^t \leftrightarrow a_2^{t-1}) \wedge (c^t \leftrightarrow c^{t-1}))) \\ \wedge (m_{t,3,1} \rightarrow ((\neg c^t) \wedge (a_1^t \leftrightarrow a_1^{t-1}) \wedge (a_2^t \leftrightarrow a_2^{t-1}) \wedge (b_1^t \leftrightarrow b_1^{t-1}) \wedge (b_2^t \leftrightarrow b_2^{t-1})))$$

Finally, define the empty condition:

$$E_t = \neg a_1^t \wedge \neg a_2^t \wedge \neg b_1^t \wedge \neg b_2^t \wedge \neg c^t$$

Thus, E_t means that all stones have been removed after round t .

Since player 1 moves at rounds 1, 3, and 5, player 1 wins exactly when the game first becomes empty at one of these rounds:

$$W = E_1 \vee (\neg E_1 \wedge \neg E_2 \wedge E_3) \vee (\neg E_1 \wedge \neg E_2 \wedge \neg E_3 \wedge \neg E_4 \wedge E_5)$$

Therefore, the final Boolean formula is

$$\Phi = I \wedge O \wedge W \wedge \left(\bigwedge_{t=1}^5 (\text{ONE}_t \wedge L_t \wedge T_t) \right)$$

This completes the QBF encoding of the winning condition for player 1.

4. (b)

By manual reasoning, the formula is true. In part (a), the quantified variables are the move variables inside each M_t . Thus, to show the formula is true, it suffices to give a winning assignment strategy for player P_1 .

For the first move, let P_1 choose

$$m_{1,3,1} = 1$$

and set all other variables in M_1 to 0. This means that player P_1 removes the only stone from pile 3. After this move, the state becomes

$$(2, 2, 0).$$

Now the two remaining nonempty piles have the same number of stones. We claim that P_1 can always maintain this property. More precisely, the existential assignments for M_3 and M_5 are chosen according to the universally chosen move of P_2 :

- if P_2 chooses $m_{t,1,1} = 1$, then P_1 chooses $m_{t+1,2,1} = 1$;
- if P_2 chooses $m_{t,1,2} = 1$, then P_1 chooses $m_{t+1,2,2} = 1$;
- if P_2 chooses $m_{t,2,1} = 1$, then P_1 chooses $m_{t+1,1,1} = 1$;
- if P_2 chooses $m_{t,2,2} = 1$, then P_1 chooses $m_{t+1,1,2} = 1$.

In each case, all other move variables in that round are set to 0, so the one-hot constraint is satisfied.

Therefore, after every move of P_2 , player P_1 makes the same move on the other nonempty pile. Hence the two piles remain equal after each move of P_1 . Since the game starts from $(2, 2, 0)$ after the first move of P_1 , this mirror strategy guarantees that whenever P_2 makes the piles unequal, P_1 immediately restores equality. As a result, P_2 can never take the last stone first, and eventually P_1 takes the last stone.

Therefore, the QBF is true, and the corresponding winning strategy is:

$$m_{1,3,1} = 1$$

for the first move, and afterwards choose the matching move on the other pile in response to player P_2 's move.

5 Binary Decision Diagram

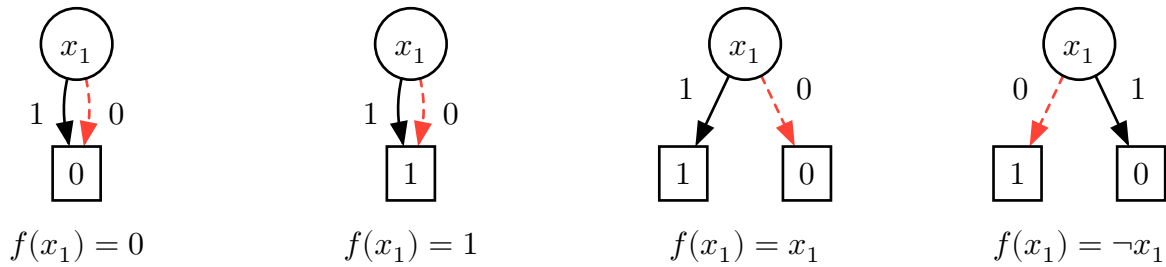
5. (a)

Let $f(x_1, x_2, \dots, x_n)$ be a any n -input Boolean function. WLOG, we can assume the variable ordering is $x_1 < x_2 < \dots < x_n$. We want to show that the ROBDD of f is unique. We prove this by induction on n .

Base case: $n = 1$. In this case, f can only be one of the following four functions:

1. $f(x_1) = 0$;
2. $f(x_1) = 1$;
3. $f(x_1) = x_1$;
4. $f(x_1) = \neg x_1$.

The ROBDD of each of these functions is unique as shown below:



Induction step: Assume the statement holds for any Boolean function for $n = 1, 2, \dots, k - 1$. We want to show that the statement also holds for $n = k$. Let $f(x_1, x_2, \dots, x_k)$ be an n -input Boolean function. By Shannon expansion, we have

$$f = x_1 f_{x_1} + \neg x_1 f_{\neg x_1}.$$

By the definition of ROBDD, the root node is decided by the first variable x_1 . The high child of the root node is the ROBDD of f_{x_1} , and the low child of the root node is the ROBDD of $f_{\neg x_1}$.

By the induction hypothesis, f_{x_1} and $f_{\neg x_1}$ are $k - 1$ input Boolean functions, hence the ROBDDs of f_{x_1} and $f_{\neg x_1}$ are unique. Therefore, the ROBDD of f is also unique, since the root node is unique and the high and low children are unique. This completes the induction step.

Thus, by induction, the ROBDD of any n -input Boolean function is unique.

5. (b)

$$f = \bigwedge_{i=1}^n (x_i \vee y_i)$$

Case 1: $O(n)$ nodes

Consider the order of variables to be $x_1 < y_1 < x_2 < y_2 < \dots < x_n < y_n$. We can construct an ROBDD of f having $O(n)$ nodes as shown below:

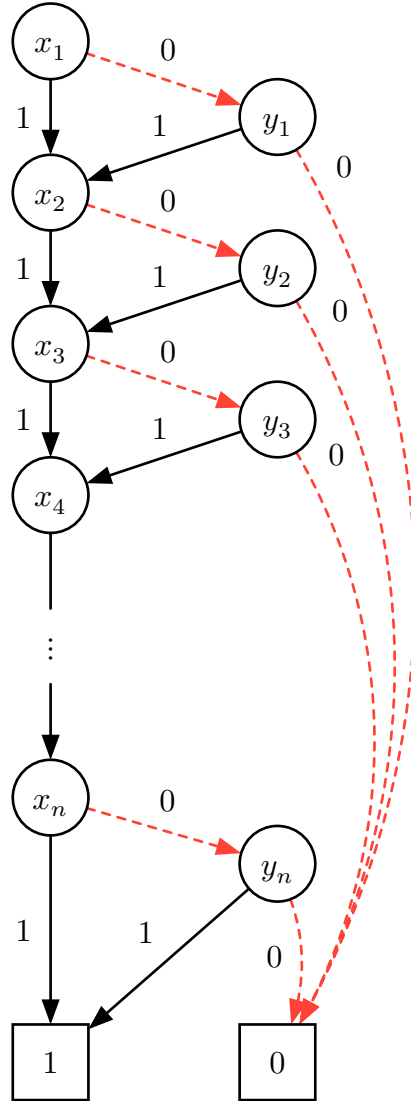


Figure 4: An ROBDD of f with $O(n)$ nodes.

Case 2: $O(2^n)$ nodes

Consider the variable order $x_1 < x_2 < \dots < x_n < y_1 < y_2 < \dots < y_n$. We show that the ROBDD of $f = \bigwedge_{i=1}^n (x_i \vee y_i)$ has $O(2^n)$ nodes.

First, consider the part of the ROBDD before any y -variable is tested. Since the variables $x_1, \dots, x_n$ are tested first, this part forms a binary decision tree of height n . Hence it contains $2^n - 1$ nodes.

Next, after all $x_1, \dots, x_n$ are assigned, the remaining function is determined by the set of indices i such that $x_i = 0$. For each assignment $(x_1, \dots, x_n) = (a_1, \dots, a_n) \in \{0, 1\}^n$, the residual function is $\bigwedge_{i:a_i=0} y_i$. Since there are only 2^n possible subsets of $\{1, \dots, n\}$, there are

at most 2^n distinct residual functions. Because an ROBDD merges identical subgraphs, the part below the x_n -level contains at most 2^n distinct subgraphs.

Therefore, the total number of nodes is at most $(2^n - 1) + 2^n = 2^{n+1} - 1 = O(2^n)$. Hence the ROBDD has $O(2^n)$ nodes.

An example of such an ROBDD with $n = 3$ is shown below:

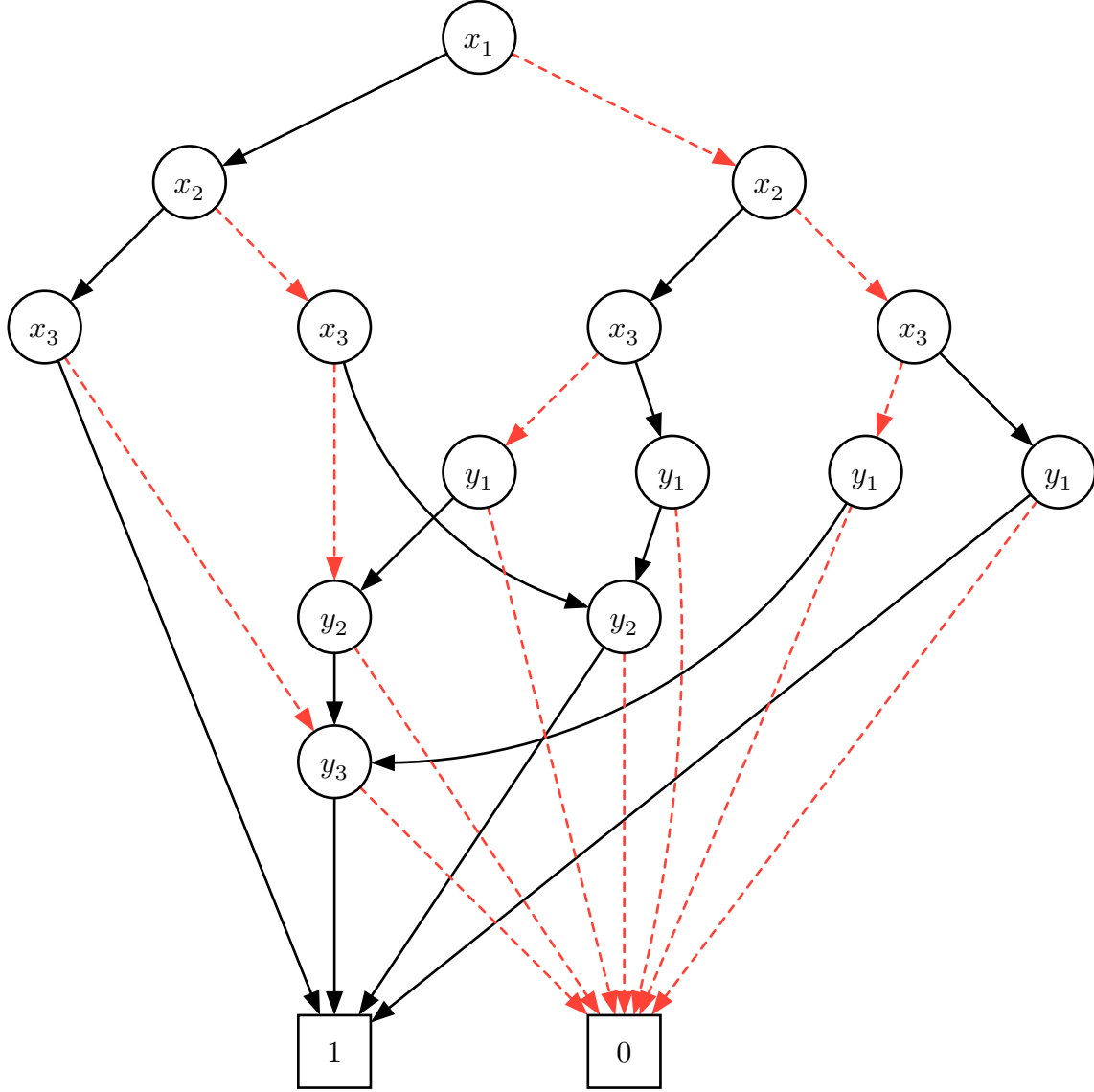


Figure 5: A ROBDD of f with $O(2^n)$ nodes. In this example, $n = 3$ and the ROBDD has 15 nodes.